

Security Audit

Rizenet Token ("RIZE") (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	10
Security Rating	12
Intended Smart Contract Behaviours	13
Code Quality	15
Audit Resources	15
Dependencies	15
Severity Definitions	16
Audit Findings	17
Centralisation	21
Conclusion	22
Our Methodology	23
Disclaimers	25
About Hashlock	26

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The T-RIZE team partnered with Hashlock to conduct a security audit of their Rizenet Governance Token (“RIZE”) smart contracts. Hashlock manually and proactively reviewed the code to ensure the project’s team and community that the deployed contracts were secure.

Project Context

Rizenet, developed by the T-RIZE Group as an Avalanche Layer 1, is a permissioned hybrid blockchain that combines the transparency of public blockchains with the security of private ones. It features a permissioned set of validators, ensuring only authorized entities validate transactions, and permissioned smart contract deployment, ensuring only authorized and identified parties can deploy smart contracts. Using the Ethereum Virtual Machine (EVM) for seamless compatibility, Rizenet is designed to unlock liquidity, enhance asset value, and provide compliant access to tokenized Real World Assets (RWAs), making it ideal for institutional investors seeking both security and scalability.

The \$RIZE token is integral to the ecosystem, governing blockchain development and facilitating payments for tokenization services on the patent-pending T-RIZE platform. This platform seamlessly integrates decentralized machine learning with a scalable blockchain infrastructure, delivering secure and transparent solutions for investors and institutions.

Rizenet drives new growth opportunities by reducing costs, expanding access to high-quality assets, and enhancing connectivity with the global financial ecosystem, positioning itself as a key enabler for value creation in tokenized assets.

To support the launch of the \$RIZE token, T-RIZE Group has committed to a fixed conversion rate for the first 12 months post-launch and will exclusively accept RIZE tokens as payment for tokenization services on the T-RIZE platform. This strategy aims



to sustain strong early demand for the token, fostering adoption and building momentum within the ecosystem.

In addition, the RIZE token ecosystem will incorporate a gamified governance layer to drive community engagement. Bonded tokens will be represented as soul-bound NFTs, providing digital proof of governance participation. This NFT system will introduce exclusive collections with rarity tiers through partnerships with brands, artists, and other projects, with limited-time or branded NFTs incentivizing users to actively participate in governance while earning collectible rewards.

Project Name: Rizenet Token (RIZE)

GitHub Commit Hash: 96eef72596a32fcdbee8a64c61f7a50d736c0fef

Compiler Version: ^0.8.26

Website: t-rize.io

Logo:



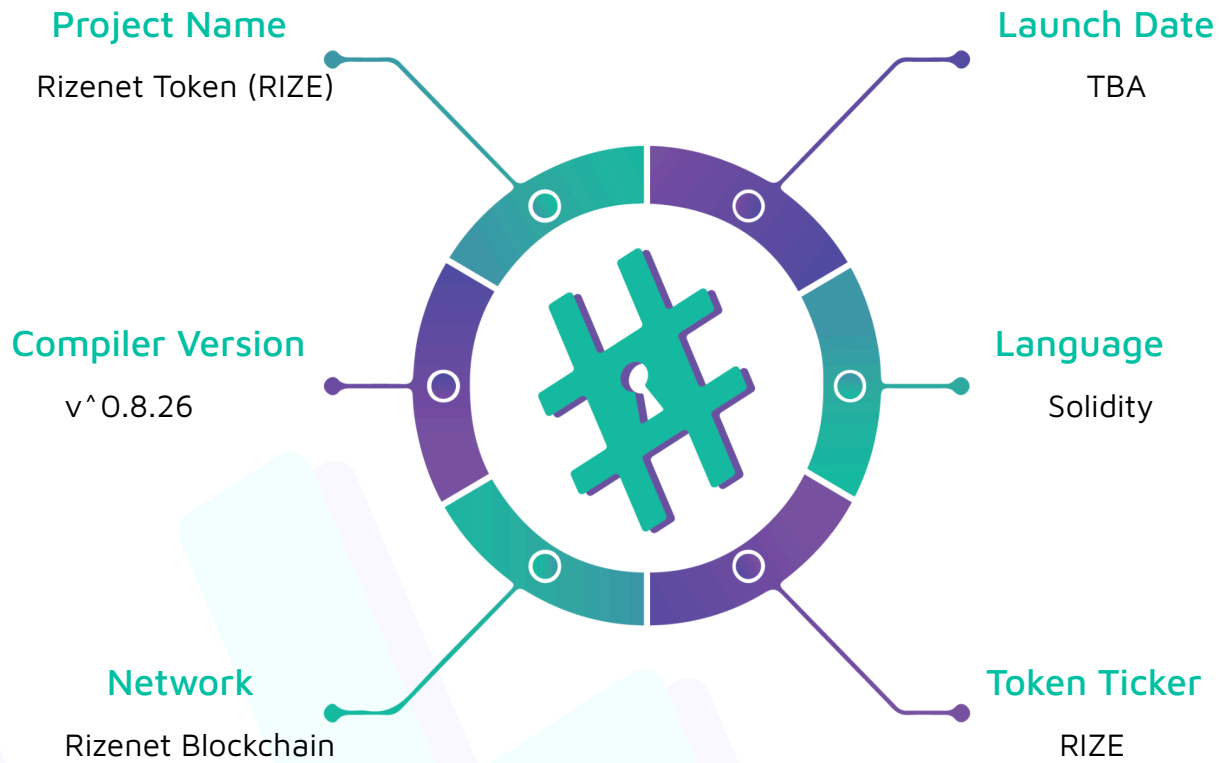
RIZE
TOKEN

POWERED BY



#Hashlock.

Hashlock Pty Ltd

Visualised Context:

Project Visuals:

T-RIZE Tokenization Platform:

Blockchain Powered Tokenization – The Next Generation of Value Creation

Our Technology streamlines access to a diverse mix of traditional and digital assets, enhances operational efficiency and significantly reduces costs. Elevate your investment strategy with our direct link between issuers and investors.

ANY ASSET

[Start for Free](#)

[Learn more](#)

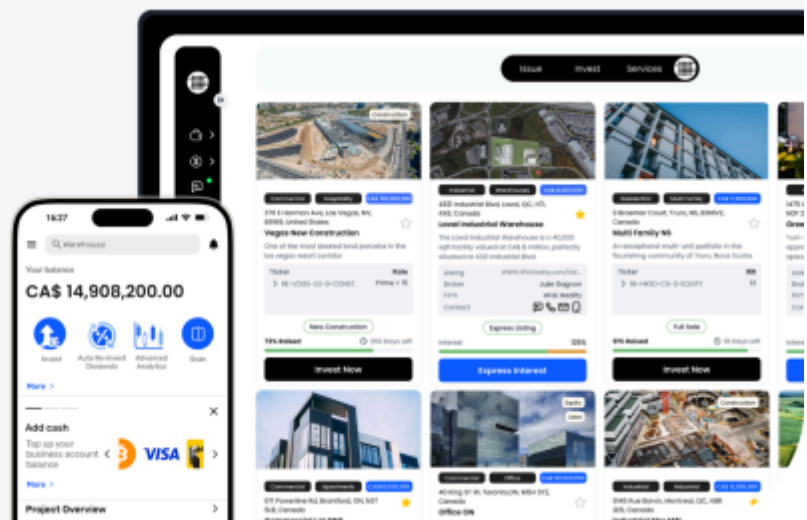
Simplifying Real-World Assets Investment Process

Unlock your asset liquidity and efficiently connect your RWA projects to global investors.

Fast Fundraising
& Enhanced Liquidity

Security
& Compliance

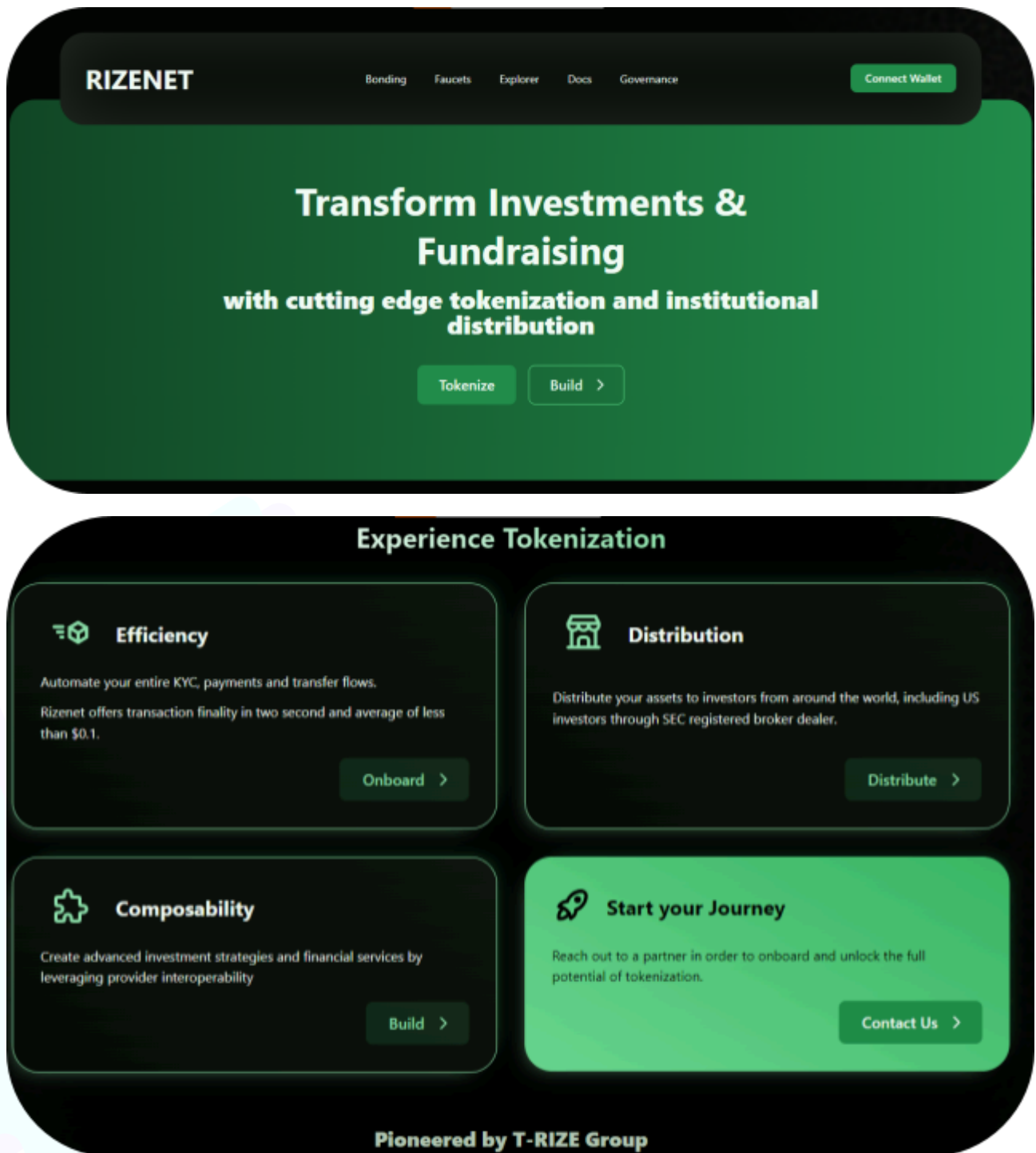
Global Reach &
Fractional Investment



#Hashlock.

Hashlock Pty Ltd

Rizenet Portal:



Rizenet & T-RIZE Documentation:

Ecosystem DocsRIZENETT-RIZE

RIZENET & T-RIZE Documentation

T-RIZE is pioneering the development of a decentralized and governable tokenization ecosystem

RIZENET DocsT-RIZE Docs

Ecosystem DocsRIZENETT-RIZE

Overview

Getting Started

Connecting to Rizenet

Invest

Token Faucet

RIZE Token

Introduction

Token Utility

Developer Resources

Network Parameters

Overview

About Rizenet

The Rizenet blockchain operates on a permissioned hybrid blockchain designed specifically for asset tokenization, built with the Avalanche L1 technology using the Ethereum Virtual Machine (EVM) as the smart contract execution engine. The chain is intricately woven into the broader T-RIZE tokenization platform to enhance functionality and integration.

Since March 2024, the Rizenet testnet is operational and under active development. Engineers are testing its capabilities, partners are providing operational feedback and the research partners from the T-RIZE Labs are working on the documentation. A significant development is the integration of university validators, which bolsters the network's integrity and security. The mainnet will be launched once all testing is completed.

Ecosystem DocsRIZENETT-RIZE

Introduction

Overview

Partnerships

T-Rize Technology

T-RIZE Labs

T-Rize Platform

Patents

Community

Overview

About T-RIZE

T-RIZE Group is transforming access to traditionally illiquid assets, such as premium real estate developments and verified carbon credits, through tokenization and enabling seamless trading. The T-RIZE brand is built on four core pillars: the T-RIZE platform, the Rizenet blockchain infrastructure, T-RIZE Labs, and institutional network access. Together, these elements create a secure, scalable, and efficient framework for tokenizing and managing high-value assets. The T-RIZE platform facilitates fractional ownership and liquidity, bridging traditional finance with decentralized technologies. For asset owners, the platform unlocks liquidity, reaches global investors, and accelerates fundraising through digital tokens. At the core of this ecosystem is the Rizenet blockchain, designed to maximize the value of tokenized assets and powered by the Rizenet governance token. T-RIZE Labs drives ongoing R&D in blockchain and decentralized machine learning, ensuring the platform's continuous innovation and growth. With strong institutional access, T-RIZE Group is committed to democratizing high-quality asset ownership and reshaping the future of investment through tokenization.

#Hashlock.

Hashlock Pty Ltd

Audit scope

We at Hashlock audited the solidity code within the Rizenet Token (RIZE) project, the scope of work included a comprehensive review of the smart contracts listed below. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

These tests primarily include the manual line-by-line analysis step and on-chain provable vulnerability testing step with Hardhat framework.

We went through an in-depth manual review of the codebase, starting with a review of common vulnerabilities such as reentrancy, interger overflow/underflow, unchecked external calls, lack of access control.

Description	T-RIZE Smart Contracts
Platform	RIZENET / Solidity
Audit Date	September, 2024
Contract 1	AccessList.sol
Contract 1 MD5 Hash	29bc6cf5b40b029cd58e5e9f3ec874d0
Contract 2	AccessRestricted.sol
Contract 2 MD5 Hash	b5e98e4294c4374c41e66677e2eb5f1c
Contract 3	BondVesting.sol
Contract 3 MD5 Hash	f557572d6c7d6f99eac4263c3b7e46ba
Contract 4	EIP3009.sol
Contract 4 MD5 Hash	db9ffebb815792068bfa305daae69f44
Contract 5	GovernanceBonding.sol
Contract 5 MD5 Hash	82dc53cc1c782012ac6e29627314f801
Contract 6	MaturityPools.sol
Contract 6 MD5 Hash	78ad45534edb8415e1fd2f4b47df75b8
Contract 7	NFTBond.sol

Contract 7 MD5 Hash	a05d9beb31cf059c132b3e2d815476d6
Contract 8	RizeToken.sol
Contract 8 MD5 Hash	b2696396141782989d040c3bd52c1c4c
Contract 9	WeightedBonds.sol
Contract 9 MD5 Hash	b4017c72e2928329a93bcb71108ce216
GitHub Commit Hash	96eef72596a32fcdbee8a64c61f7a50d736c0fef

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section.

We initially identified some minor vulnerabilities that have since been addressed.

Hashlock found:

2 Low-severity vulnerabilities

4 QA's

2 Gas Optimizations

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Behaviours

Claimed Behaviour	Actual Behaviour
AccessList.sol Contains addresses of system contracts and roles.	Contract achieves this functionality.
AccessRestricted.sol Checks if the caller is a committee and a bridge.	Contract achieves this functionality.
BondVesting.sol Tracks vesting of tokens using a linear unlock: - Tokens are fully vested until the cliff ends - Tokens are unlocked linearly from the cliff until the end of the vesting period - Tokens are fully unlocked past the vesting	Contract achieves this functionality.
EIP3009.sol Internal implementation for gas-abstracted transfers.	Contract achieves this functionality.
GovernanceBonding.sol Allows committee to: - Define the time required between breaking the bond and calling the release function - Add a pool - Update the parameters of a pool - Revoke vested tokens of a bond - Updates the vesting schedule - Add/Remove migrator	Contract achieves this functionality.
MaturityPools.sol Contains information about the different pools and their weights.	Contract achieves this functionality.

NFTBond.sol NFT to represent the governance bonds.	Contract achieves this functionality.
RizeToken.sol WETH9 equivalent on Rizenet, but a standard ERC20 on other chains.	Contract achieves this functionality.
WeightedBonds.sol Custody tokens and stores bond related information.	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts for the Rizenet Token (RIZE) project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Rizenet Token (RIZE) smart contract code in the form of GitHub access.

As mentioned above, code parts are well-commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments help understand the overall architecture of the protocol.

Dependencies

Per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry-standard open-source projects.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies

Audit Findings

Low

[L-01] RizeToken#_withdraw - Deprecated Ether transfer function

Description

The `_withdraw` function uses the `.transfer` function to transfer native coins.

The Istanbul update made some changes to the EVM, which made the `.transfer` function deprecated for the ETH transfer.

Recommendation

Use the `.call` function for the ETH transfer and check the return value.

Status

Resolved

[L-02] RizeToken#setFeesBps - Lack of emitting an event

Description

The `setFeesBps` function is missing an event when a key storage is updated.

Recommendation

Add a relevant event in the function.

Status

Resolved

QA

[Q-01] **AccessList#setGovernanceBonding, setBonds, setToken, setTreasury**

- Missing zero address checks

Description

The `setGovernanceBonding`, `setBonds`, `setToken`, and `setTreasury` functions are missing zero address checks for the input parameters values.

Recommendation

Add checks if the input parameters values are zero.

Status

Resolved

[Q-02] **WeightedBonds** - Unused struct

Description

The `BondWithId` struct is not used at all in the contract.

Recommendation

Remove the struct.

Status

Resolved

[Q-03] **WeightedBonds** - Readability improving

Description

The `_releaseWarmup` is set to `7 * 24 * 60 * 60` but it reduces the readability.

Recommendation

Replace `7 * 24 * 60 * 60` with `7 days`.

Status

Resolved

[Q-04] MaturityPools - Unnecessary assignment**Description**

The `_poolId` is set to 0 again even if its default value is 0 as standard.

Recommendation

Avoid assigning to 0 when declaring.

Status

Resolved

Gas

[G-01] AccessRestricted, WeightedBonds - `_accessList` and `_govBond` variables could be made immutable

Description

The `_accessList` variable in the `AccessRestricted` contract is only set in the constructor.

Same for the `_govBond` variable in the `WeightedBonds` contract.

Recommendation

Make the variables immutable.

Status

Resolved

[G-02] WeightedBonds#release - `release.length` is read multiple times

Description

In the `release` function, `release.length` is read at every iteration from storage.

Recommendation

Cache `release.length` into a memory variable.

Status

Resolved

Centralisation

The Rizenet Token (RIZE) project values security and utility over decentralisation.

The Rizenet Token (RIZE) and governance contracts relies on an address labelled as “committee” with the power to:

- Configure governance pools with arbitrary parameters
- Break soul binding spell on the governance NFT
- Update the time required to unlock bonded tokens from 0 to 31 days
- Clawback vested tokens
- Decrease vesting terms
- Configure bridge contracts with the ability to mint RIZE ERC20 on non-native chains

These centralization points can be mitigated by using a multi-signature wallet with a timelock contract.

The committee cannot blacklist or transfer tokens on behalf of others.



Centralised

Decentralised

Conclusion

After Hashlocks analysis, the Rizenet Token (RIZE) project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au

#Hashlock.



#Hashlock.

Hashlock Pty Ltd